

Algorithms - CheatSheet

IN BA4 - Ola Nils Anders Svensson

Notes by Ali EL AZDI

This is a cheat sheet for the Algorithms midterm exam. For suggestions, contact me on Telegram ([elazdi_al](#)) or via EPFL email (ali.elazdi@epfl.ch).

March 25th, 2025

Master Theorem If $T(n) = aT(\frac{n}{b}) + f(n)$, $a \geq 1$, $b > 1$, and $f(n)$ asymptotically positive. Case 1: If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$. Case 2: If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$. Case 3: If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and if $a f(\frac{n}{b}) \leq c f(n)$ for some $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. Common case - If $f(n) = \Theta(n^d)$ for some exponent d: - If $\frac{d}{\log b} < 1$ (or $d < \log_b a$), then $T(n) = \Theta(n^d)$. - If $\frac{d}{\log b} = 1$ (or $d = \log_b a$), then $T(n) = \Theta(n^d \log n)$. - If $\frac{d}{\log b} > 1$ (or $d > \log_b a$), then $T(n) = \Theta(n^{\log_b a})$.	
Queue Operations Queue-Empty(Q): Time: $O(1)$, Auxiliary Space: $O(1)$ 1. Returns TRUE if the queue is empty ($Q.head = Q.tail$). 2. Returns FALSE otherwise. Enqueue(Q, x): Time: $O(1)$, Auxiliary Space: $O(1)$ 1. Adds element x to the rear of queue Q . 2. $Q[Q.tail] = x$ 3. $Q.tail = Q.tail + 1$ (or wrap around if using circular array) Dequeue(Q): Time: $O(1)$, Auxiliary Space: $O(1)$ 1. If Queue-Empty(Q), return error "underflow". 2. Otherwise, remove and return the element at the front. 3. $x = Q[Q.head]$ 4. $Q.head = Q.head + 1$ (or wrap around) 5. Return x Queue Implementation: 1. Q.head: Index of the front element 2. Q.tail: Index where next element will be inserted 3. In a circular array, indices wrap around 4. Leave one slot empty to distinguish full/empty states Overall Space Complexity: $O(n)$ for a queue of capacity n	
Merge Sort 1. Divide: Split the array evenly into two smaller subarrays, and continue dividing recursively. 2. Sort (Recursively): Apply merge sort recursively on each subarray until each has only one element (base case). MERGE-SORT(A, p, q, r) if $p < r$ $q = \lfloor (p+r)/2 \rfloor$ MERGE-SORT(A, p, q) MERGE-SORT(A, q+1, r) MERGE(A, p, q, r) 3. Merge: Combine the two sorted subarrays into a single sorted array: a) Initializing pointers at the start of each subarray. b) Comparing the elements pointed to, and appending the smaller one into a new array. c) Advancing the pointer in the subarray from which the element was chosen. d) Repeating this process until all elements in both subarrays are merged into the sorted array. Merge Cost Complexity: $O(n)$ per merge operation. Time Complexity: $O(n \log n)$ Space Complexity: $O(n)$	
Priority Queue Maintains a dynamic set of elements with associated priority values (keys). Maximum(S): Return element of S with highest priority (return $A[1]$, complexity $O(1)$) Insert(S, x): Insert element x into set S 1. Increment the heap size 2. Insert a new node in the last position in the heap, with key $-\infty$ 3. Increase the $-\infty$ value to key using Heap-Increase-Key Extract-Max(S): Remove and return element of S with highest priority 1. Make sure heap is not empty 2. Make a copy of the maximum element (the root) 3. Make the last node in the tree the new root 4. Re-heapify the heap, with one fewer node 5. Return the copy of the maximum element Increase-Key(S, x, k): Increase the value of element x 's key to the new value k 1. Make sure key $\geq A[i]$ 2. Update $A[i]$'s value to key 3. Traverse the tree upward comparing new key to the parent and swapping if necessary Time Complexity: Insert, Extract-Max, Increase-Key: $O(\log n)$ Maximum: $O(1)$ Space Complexity: $O(n)$	
Stack Operations Stack-Empty(S): Time: $O(1)$, Auxiliary Space: $O(1)$ 1. Returns TRUE if the stack is empty. 2. Returns FALSE otherwise. Push(S, x): Time: $O(1)$, Auxiliary Space: $O(1)$ 1. Adds element x to the top of stack S . 2. Increments the stack pointer. Pop(S): Time: $O(1)$, Auxiliary Space: $O(1)$ 1. If Stack-Empty(S), return error "underflow". 2. Otherwise, remove and return the top element. 3. Decrements the stack pointer. Stack Implementation: 1. Elements are stored in a simple array 2. S.top: Index of the topmost element 3. An empty stack has S.top = 0 or S.top = -1 (implementation dependent) Overall Space Complexity: $O(n)$ for a stack of size n	
Dynamic Programming Problem: Optimal solutions to overlapping subproblems Fibonacci Sequence: Top-Down (Memoization): 1. Create memo array $F[0..n]$ initialized to NIL. 2. Base cases: $F[0] = 0, F[1] = 1$ 3. Recursive with memo: - Return $F[n]$ if already computed - Otherwise compute $F[n] = F[n-1] + F[n-2]$ - Store result in $F[n]$ and return Bottom-Up (Tabulation): 1. Create array $F[0..n]$ 2. Base cases: $F[0] = 0, F[1] = 1$ 3. For $i = 2$ to n : - Compute $F[i] = F[i-1] + F[i-2]$ 4. Return $F[n]$ Time Complexity: $O(n)$ Space Complexity: $O(n)$ (includes input and memo array)	Cut-Rod Problem: Find optimal way to cut rod to maximize revenue Top-Down (Memoization): 1. Create memo array $r[0..n]$ with $r[0] = 0$ 2. For uncalculated $r[j]$, compute: $r[j] = \max_{1 \leq i \leq j} (p[i] + r[j-i])$ 3. Return $r[n]$ MEMOIZED-CUT-ROD-AUX(p, n, r) if $r[n] \geq 0$ return $r[n]$ if $n = 0$ $q = 0$ else $q = -\infty$ for $i = 1$ to n do $q = \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n-i, r))$ $r[n] = q$ return q MEMOIZED-CUT-ROD(p, n) let $r[0..n]$ be a new array let $r[i] = 0$ to n do $r[i] = \text{MEMOIZED-CUT-ROD-AUX}(p, n, r)$ return r Bottom-Up (Tabulation): 1. Create array $r[0..n]$ with $r[0] = 0$ 2. For $j = 1$ to n : - Compute $r[j] = \max_{1 \leq i \leq j} (p[i] + r[j-i])$ 3. Return $r[n]$ EXTENDED-BOTTOM-UP-CUT-ROD(p, n) let $r[0..n]$ and $s[0..n]$ be new arrays $r[0] = 0$ for $j = 1$ to n do $q = -\infty$ for $i = 1$ to j do if $q < p[i] + r[j-i]$ $q = p[i] + r[j-i]$ $s[j] = i$ $r[j] = q$ return r, s Time Complexity: $O(n^2)$ Space Complexity: $O(n)$ (includes input prices and array)

Optimal Binary Search Tree Problem: Construct a BST with minimum expected search cost given access probabilities

Optimal-BST Algorithm: - Input: Keys $K_1, \dots, K_n$, probabilities $p_1, \dots, p_n$ for successful searches

- Optional: Probabilities $q_0, \dots, q_n$ for unsuccessful searches

- Create tables $e[1 \dots n+1, 0 \dots n]$ for expected costs

- Create $w[i, j]$ for sum of probabilities from i to j

- Create $root[i, j]$ to record optimal roots

- Fill tables bottom-up by increasing subproblem size

- For each subproblem, try all possible roots and pick the minimum cost

- Formula: $e[i, j] = \min_{i \leq r \leq j} \{ e[i, r-1] + e[r+1, j] + w[i, j] \}$

- The root of the overall optimal tree is in $root[1, n]$

let $e[1 \dots n+1, 0 \dots n]$, $w[1 \dots n+1, 0 \dots n]$, and $root[1 \dots n, 1 \dots n]$ be new tables

```
for i = 1 to n+1 do
  e[i, i-1] ← 0
  w[i, i-1] ← 0
for i = 1 to n do
  for l = 1 to n-l+1 do
    j ← i+l-1
    e[i, j] ← ∞
    w[i, j] ← w[i, j-1] + p_j
    for r = i to j do
      t ← e[i, r-1] + e[r+1, j] + w[i, j]
      if t < e[i, j]
        e[i, j] ← t
        root[i, j] ← r
    return e, root
```

Complexity:

- Time: $O(n^3)$

- Space: $O(n^2)$ for tables e , w , and $root$

- The algorithm computes optimal costs for all possible subtrees

Matrix Chain Multiplication:

Find optimal parenthesization to minimize multiplications

Bottom-Up (Tabulation):

1. Create table $m[1 \dots n, 1 \dots n]$ with $m[i, i] = 0$

- $m[i, j]$ stores minimal cost of multiplying matrices i through j

2. For $l = 2$ to n (chain length):

3. For $i = 1$ to $n-l+1$:

- Set $j = i+l-1$

- Compute $m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \}$

- Store k in $s[i, j]$ that achieved minimum cost

4. Return $m[1, n]$

```
MATRIX-CHAIN-ORDER(p)
n ← p.length - 1
let m[1..n, 1..n] and s[1..n, 1..n] be new tables
for i = 1 to n do
  m[i, i] ← 0
for l = 2 to n do // l is the chain length
  for i = 1 to n-l+1 do
    j ← i+l-1
    m[i, j] ← ∞
    for k = i to j-1 do
      q ← m[i, k] + m[k+1, j] + p[i-1] p_k p_j
      if q < m[i, j]
        m[i, j] ← q
        s[i, j] ← k
    return m, s
```

Complexity:

- Time: $O(n^3)$

- Space: $O(n^2)$ (includes matrix dimensions and tables)

- Table $s[i, j]$ stores index of last matrix in

first parenthesized group

Linked List

Linear data structure where each node contains:

1. **key/data:** The value stored in the node

2. **next:** A pointer to the next node in the sequence

3. **prev:** A pointer to the previous node (in doubly linked lists)

List-Insert (insert a new node at the List-Delete

beginning)

1. Create a new node with the given 1. Find the node to be deleted (may

key.

2. Set the next pointer of the new node 2. If the node is the head, update the

head pointer to the next node.

3. If implementing a doubly linked list, 3. Otherwise, update the next pointer

set the prev pointer of the current head of the previous node to skip the node

to the new node.

4. Update the head pointer to point to 4. For doubly linked lists, also update

the prev pointer of the next node.

5. If the list was empty, update the tail 5. Free the memory allocated for the

pointer as well.

LIST-INSERT(L, x)

x.next ← L.head

If L.head ≠ nil

L.head.prev ← x

L.head ← x

x.prev ← NIL

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Time Complexity: $O(n^2)$ for finding

the node, $O(1)$ for deletion

Space Complexity: $O(1)$

List-Search (find a node with a given key)

1. Start from the head of the linked list.

2. Traverse the list by following the next

pointers.

3. Compare each node's key with the target

key.

4. Return the node if the key is found.

5. Return NULL if the end of the list is reached without finding the key

Time Complexity: $O(n)$

Space Complexity: $O(1)$

LIST-SEARCH(L, k)

x ← L.head

while x ≠ nil ∧ x.key ≠ k do

x ← x.next

return x

Longest Common Subsequence

Problem: Find the longest subsequence common to two sequences

LCS-Length:

- Build tables for length $c[0 \dots m, 0 \dots n]$ and direction

$b[1 \dots m, 1 \dots n]$

- Initialize first row and column to zeros

- For each cell (i, j) in the table:

If characters match, take diagonal value + 1

Otherwise, take maximum from above or left

- Table $c[m, n]$ contains the LCS length

- Table b records decisions for reconstruction

LCS-LENGTH(X, Y, m, n)

let $b[1 \dots m, 1 \dots n]$ and $c[0 \dots m, 0 \dots n]$ be new tables

for $i = 1$ to m do

$c[i, 0] \leftarrow 0$

for $j = 0$ to n do

$c[0, j] \leftarrow 0$

for $i = 1$ to m do

for $j = 1$ to n do

if $x_i = y_j$

$c[i, j] \leftarrow c[i-1, j-1] + 1$

$b[i, j] \leftarrow \nwarrow$

else if $c[i-1, j] \geq c[i, j-1]$

$c[i, j] \leftarrow c[i-1, j]$

$b[i, j] \leftarrow \uparrow$

else

$c[i, j] \leftarrow c[i, j-1]$

$b[i, j] \leftarrow \leftarrow$

return c, b

Time Complexity: $O(mn)$ for two sequences of lengths m and n

Space Complexity: $O(mn)$ (includes input sequences and tables)

Space can be optimized to $O(m+n)$ if only length is needed

Print-LCS takes $O(m+n)$ time to reconstruct the solution

Print-LCS:

- Recursively trace back through direction table b

- Follow diagonal arrows and print characters

- Skip cells with up or left arrows

- Stops when reaching first row or column

```
PRINT-LCS(b, X, i, j)
if i = 0 ∨ j = 0 return
if b[i, j] == ↖
  PRINT-LCS(b, X, i-1, j-1)
print x_i
else if b[i, j] == ↑
  PRINT-LCS(b, X, i, j-1)
else
  PRINT-LCS(b, X, i, j-1)
```

Example Recording of Solution:

	1	2	3	4	5	6	7
i	0	0	0	0	0	0	0
1	0	0	↗	0	↗	↖	↖
2	0	0	↖	↖	↖	↖	↖
3	0	0	↖	↖	↖	↖	↖
4	0	↖	↖	↖	↖	↖	↖
5	0	↖	↖	↖	↖	↖	↖
6	0	↖	↖	↖	↖	↖	↖

Depth-First Search (DFS)

Goal:

Explore a graph by going as deep as possible along each

path before backtracking (like exploring a maze).

How it works:

1 - Initialize all vertices as WHITE (unvisited).

2 - For each WHITE vertex, start DFS-Visit:

3 - Mark current vertex as GRAY (being processed),

record discovery time

4 - Recursively visit each WHITE neighbor (ignore

GRAY/BLACK).

5 - When done with all neighbors, mark vertex as

BLACK (finished), record finish time.

Key Data: Discovery and finish times are kept for each

node in DFS. These times help determine ancestor-

descendant relationships and can be used for topological

sorting.

DFS-Visit Time Complexity: $O(E)$

DFS-Visit Space Complexity: $O(V)$

Time Complexity: $O(V+E)$

Space Complexity: $O(V)$

DFS(G)

for all $u \in G.V$ do

$u.d \leftarrow \text{time} + 1$

$u.\text{color} \leftarrow \text{WHITE}$

time ← 0

for all $u \in G.V$ do

for all $v \in G.\text{Adj}[u]$ do

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)

$u.\text{color} \leftarrow \text{BLACK}$

time ← time + 1

$u.f \leftarrow \text{time}$ // finish u

DFS-VISIT(G, u)

(u, v)

if $v.\text{color} = \text{WHITE}$

DFS-VISIT(G, v)